NEW Self-hosted Retool 4.0, 3.334, and 3.300 stable updates

System architecture of self-hosted Retool deployments

Learn about Retool's self-hosted architecture.

RETOOL 4.23.0 EDGE ENTERPRISE

Organizations that have on-premise requirements can deploy the Retool platform on their own infrastructure. Each deployment instance uses a distributed set of containers with services for different functions. These work together to securely run the platform within your VPN or VPC.

> Self-hosted deployments support classic apps out of the box. To build apps using AI and natural language prompts, you must enable services for the new app builder in your Helm values and have blob storage configured and your Retool ingress in place. Set `rr.enabled: true` to enable the `js-executor`, `agent-sandbox`, and `rr.gitServer` services together. Set `mcp.enabled: true` separately to enable the MCP service.

Containers and services

A standard deployment instance includes the following services. Most run as a single container. The exception is `agent-sandbox`, which consists of multiple cooperating containers. Refer to the agent-sandbox section for the full component breakdown. The `SERVICE_TYPE` environment variable values determine which services certain containers run.

CONTAINER	IMAGE	REPOSITORY	SERVICES

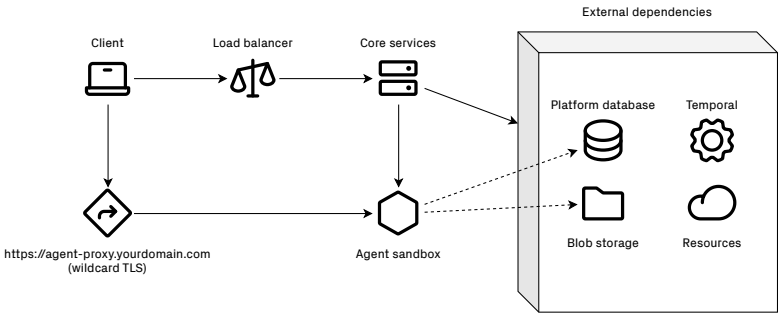
api	`backend`	tryretool/backend	`MAIN_BACKEND`
			`DB_CONNECTOR`
			`DB_SSH_CONNECTOR`

jobs-runner	`backend`	tryretool/backend	`JOBS_RUNNER`

CONTAINER	IMAGE	REPOSITORY	SERVICES
workflows-worker	`backend`	tryretool/backend	`WORKFLOW_TEMPORAL_WORKER`
workflows-backend	`backend`	tryretool/backend	`WORKFLOW_BACKEND` `DB_CONNECTOR` `DB_SSH_CONNECTOR`
agent-worker	`backend`	tryretool/backend	`AGENT_TEMPORAL_WORKER`
agent-eval-worker	`backend`	tryretool/backend	`AGENT_EVAL_TEMPORAL_WORKER`
code-executor	`code-executor-service`	tryretool/code-executor-service	No service type.
js-executor	`js-executor-service`	tryretool/js-executor-service	No service type.
agent-sandbox	`agent-sandbox-service`	tryretool/agent-sandbox-service	No service type. Consists of multiple containers (refer to agent-sandbox).

Architecture overview





54%

api

The **api** container manages the core functionality for a Retool deployment instance, such as:

- Frontend interactions (e.g., building apps and workflows).
- Hosting apps and workflows.
- Managing users and permissions.

api runs the following services:

```
MAIN_BACKEND
```

retool.example

The core service for a Retool deployment instance. It handles most logic for frontend interactions.

`DB_CONNECTOR`

Handles query requests to resources (databases, APIs, etc.).

`DB_SSH_CONNECTOR`

Handles query requests to resources that use SSH tunnels. Operates the same way as `DB_CONNECTOR` but exclusively for resources configured with SSH proxies.

Network ingress and egress

api requires ingress from client interactions, such as loading an app in a browser. These are typically handled by a load balancer (e.g., nginx) which proxies the requests.

The following table displays ingress and egress requirements for **api**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
code-executor	✗	✓
postgres (platform database)	✗	✓
retooldb-postgres	✗	✓
Temporal	✗	✓
Resources	✗	✓

Replicas

You can replicate **api** as you scale to manage higher volumes of workflow traffic.

RESOURCE QUERY TRACE retool.example

jobs-runner

The **jobs-runner** container manages background tasks, runs database migrations for Retool version upgrades, and Source Control. It runs the following service:

`JOBS_RUNNER`

Performs database migrations and manages Source Control.

Network ingress and egress

The following table displays ingress and egress requirements for **jobs-runner**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
postgres	✕	✓

Replicas

You must not replicate **jobs-runner**. It performs tasks and migrations that must be performed as a single container.

workflows-worker

The **workflows-worker** container continuously polls the Temporal Cluster for tasks required to either start or execute blocks within a workflow. It makes requests to **code-executor** to execute blocks and process results, then reports back to Temporal to continue or complete workflow execution.

workflows-worker runs the following service.

`WORKFLOW_TEMPORAL_WORKER`

Polls the Temporal Cluster for tasks required to start or execute workflow blocks.

Network ingress and egress

The following table displays ingress and egress requirements for **workflows-worker**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
code-executor	✗	✓
postgres (platform database)	✗	✓
retooldb-postgres	✗	✓
Temporal	✗	✓
Resources	✗	✓

Replicas

You can replicate **workflows-worker** as you scale to manage higher volumes of workflow traffic.

workflows-backend

The **workflows-backend** container receives and processes requests from **code-executor** to process workflows. It also handles the upload of workflows logs, block results, and run status updates.

`WORKFLOW_BACKEND`

Handles query, retry, and asynchronous logic.

`DB_CONNECTOR`

Handles query requests to resources (databases, APIs, etc.).

`DB_SSH_CONNECTOR`

Handles query requests to resources that use SSH tunnels. Operates the same way as `DB_CONNECTOR` but exclusively for resources configured with SSH proxies.

« Network ingress and egress retool.example

The following table displays ingress and egress requirements for **workflows-backend**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
code-executor	✓	✓
postgres (platform database)	×	✓
retooldb-postgres	×	✓
Temporal	×	✓
Resources	×	✓

Replicas

You can replicate **workflows-backend** as you scale to manage higher volumes of workflow traffic.

agent-worker

The **agent-worker** container continuously polls the Temporal cluster for pending agent runs to execute. It makes requests to code-executor to execute the agent logic, call tools and process results, and then it reports back to Temporal to continue or complete agent execution.

Network ingress and egress

The following table displays ingress and egress requirements for **agent-worker**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
code-executor	×	✓
postgres (platform database)	×	✓
retooldb-postgres	×	✓
Temporal	×	✓
Resources	×	retool.example ✓

Replicas

You can replicate **agent-worker** as you scale to manage higher volumes of workflow traffic.

agent-eval-worker

The **agent-eval-worker** continuously polls the Temporal Cluster for pending agent eval runs. When an eval is running, the **agent-eval-worker** makes calls from Postgres to the LLM provider configured in the test case, and compares the output to the expected value.

Network ingress and egress

The following table displays ingress and egress requirements for **agent-eval-worker**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
code-executor	×	✓
postgres (platform database)	×	✓
Temporal	×	✓
Resources	×	✓

Replicas

You can replicate **agent-eval-worker** as you scale to manage higher volumes of workflow traffic.

code-executor

The code-executor container executes single block runs from the Workflows IDE, multiple blocks as part of a Workflow execution, and any arbitrary user code written in a Workflow. It executes JavaScript and Python code directly, or makes requests to **workflows-backend** to run resource queries against databases, APIs, etc.

By default, the **code-executor** uses nsjail to sandbox code execution. nsjail supports three security modes, from most to least secure:

1. **Custom seccomp profile** (recommended): grants only the specific permissions nsjail requires. Refer to Run code executor in unprivileged mode for setup.

2. **Privileged mode**: not recommended. Grants broad host access including `CAP_SYS_ADMIN`, which is unnecessary for sandboxing.
3. **Neither**: nsjail sandboxing disabled. Least secure; use only if your environment does not support seccomp profiles or privileged containers.

Network ingress and egress

The following table displays ingress and egress requirements for **code-executor**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
api	✓	×
workflows-backend	×	✓
workflows-worker	✓	×

Replicas

You can replicate **code-executor** as you scale to manage higher volumes of workflow traffic.

js-executor

The **js-executor** container provides a sandboxed JavaScript execution environment for backend functions. It does not require privileged container access, but does require a custom Linux seccomp profile on each cluster node (the same requirement as the agent sandbox). For seccomp profile details, refer to Custom code execution security.

Network ingress and egress

The following table displays ingress and egress requirements for **js-executor**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
api	✓	×

Replicas

You can replicate **js-executor** to handle higher volumes of traffic.

The **agent-sandbox** manages the app editing experience. Each active builder session runs in an isolated sandbox pod with a web server for app previews and a WebSocket connection to the builder's browser. It uses gVisor for secure isolation without requiring root access.

The agent sandbox consists of several cooperating components:

COMPONENT	CPU	MEMORY	SCALING
`agent-sandbox-controller`	¼ core	256 MiB	Minimal
`agent-sandbox-proxy`	¼ core	256 MiB	Minimal
`r2-worker`	1 core	2 GiB	Minimal
`agent-sandbox-job` pods	1 core	2 GiB	~1 pod per active editor session

A prewarm pool of idle sandbox pods (default: 5) reduces startup latency for new editing sessions. This is configurable down to 0.

Network ingress and egress

By default, **agent-sandbox-proxy** routes sandbox traffic through your existing Retool domain using same-origin networking — no separate subdomain, wildcard DNS, or additional TLS certificate required. To use a dedicated proxy subdomain instead, set `rr.agentSandbox.frontendWsProxyDomain` in your Helm values.

The following table displays ingress and egress requirements for **agent-sandbox**.

CONTAINER	NETWORK INGRESS	NETWORK EGRESS
Browser (via main ingress)	✓	×
api	✓	✓
Kubernetes API (controller)	×	✓
postgres (platform database)	×	✓

retool.example

Replicas

Sandbox **job pods** auto-scale based on active builder sessions — each open editing session requires approximately one pod. The controller, proxy, and r2-worker do not need to be replicated.

postgres

postgres manages the platform database, which stores your instance's apps, resources, users, permissions, and internal data. All deployment instances contain a default `postgres` container for testing purposes. You must externalize this database before using the deployment instance in production.

When configuring an external database, you must:

- Enable the uuid-ossp module.
- Use the Read Committed isolation level.

Retool creates a number of tables in the database, such as:

TABLE	DESCRIPTION
`app_observability_provider_configs`	Observability setup and streaming details.
`audit_trail_events`	Events that feed audit logs on user instances with the `/audit` endpoint.
`grid_managed_cluster_resources`	Retool Database when linked to an external PostgreSQL database.
`grid_managed_clusters`	Retool Database when linked to an external PostgreSQL database.
`organizations`	Spaces within the organization.
`page_saves`	Changes made to specific apps since they were created.
`resources`	Resources, including full setup and configuration information.
`users`	All enabled and disabled users.

TABLE	DESCRIPTION
`user_task_instance`	Workflows that include user task blocks.

> Do not directly modify Retool table data. Doing so can damage the instance.

ALL STORAGE DATABASE TABLES

retooldb-postgres

You can optionally configure your deployment to use Retool Database with a separate PostgreSQL database. You can use `retooldb-postgres` to get started but Retool recommends you migrate to an externally hosted database for use in production.

mcp

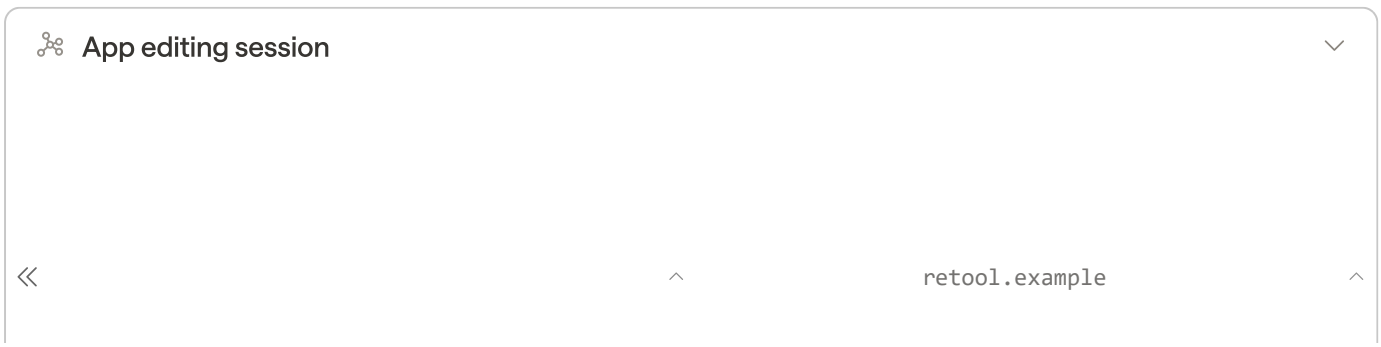
The **mcp** service runs a Model Context Protocol (MCP) server that exposes your Retool resources and tools to MCP-compatible AI clients. It is disabled by default and enabled independently of the new app builder services master switch.

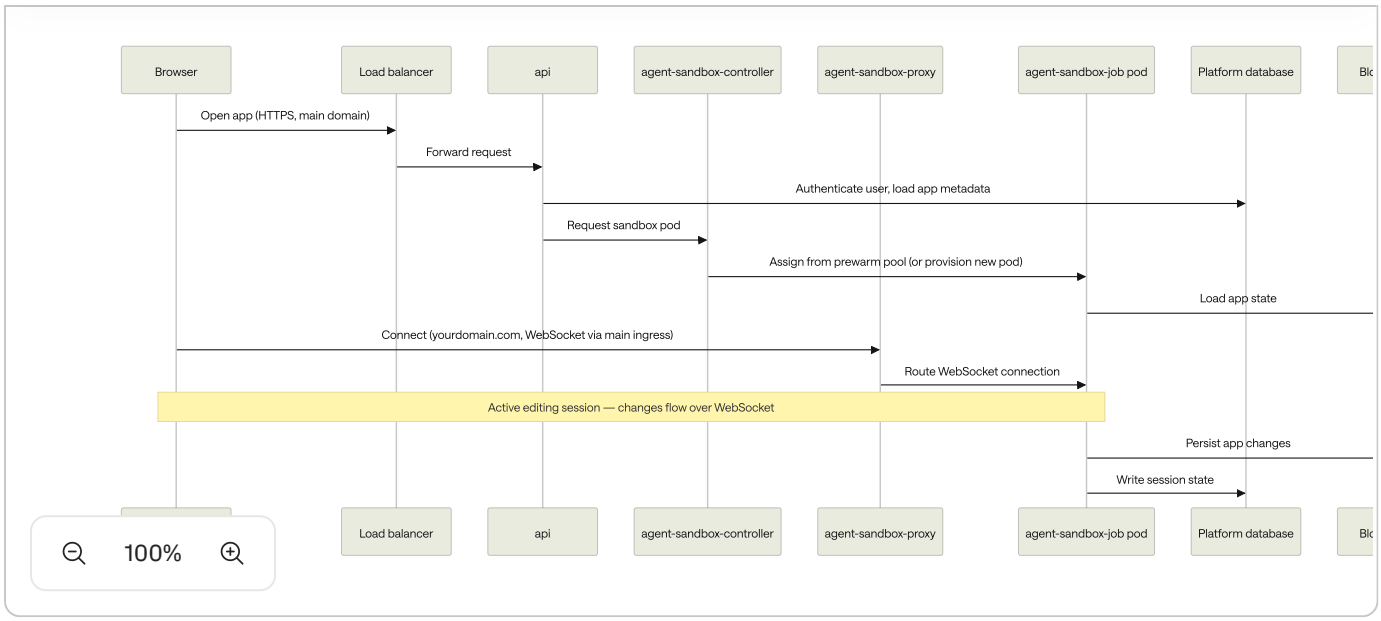
To enable the MCP service, set `mcp.enabled: true` in your Helm values. This setting is top-level and independent of `rr.enabled` — you can enable MCP without enabling any other new app builder service. The service routes through your existing Retool ingress at `<your-domain>/mcp/*`. No additional subdomain or wildcard certificate is required.

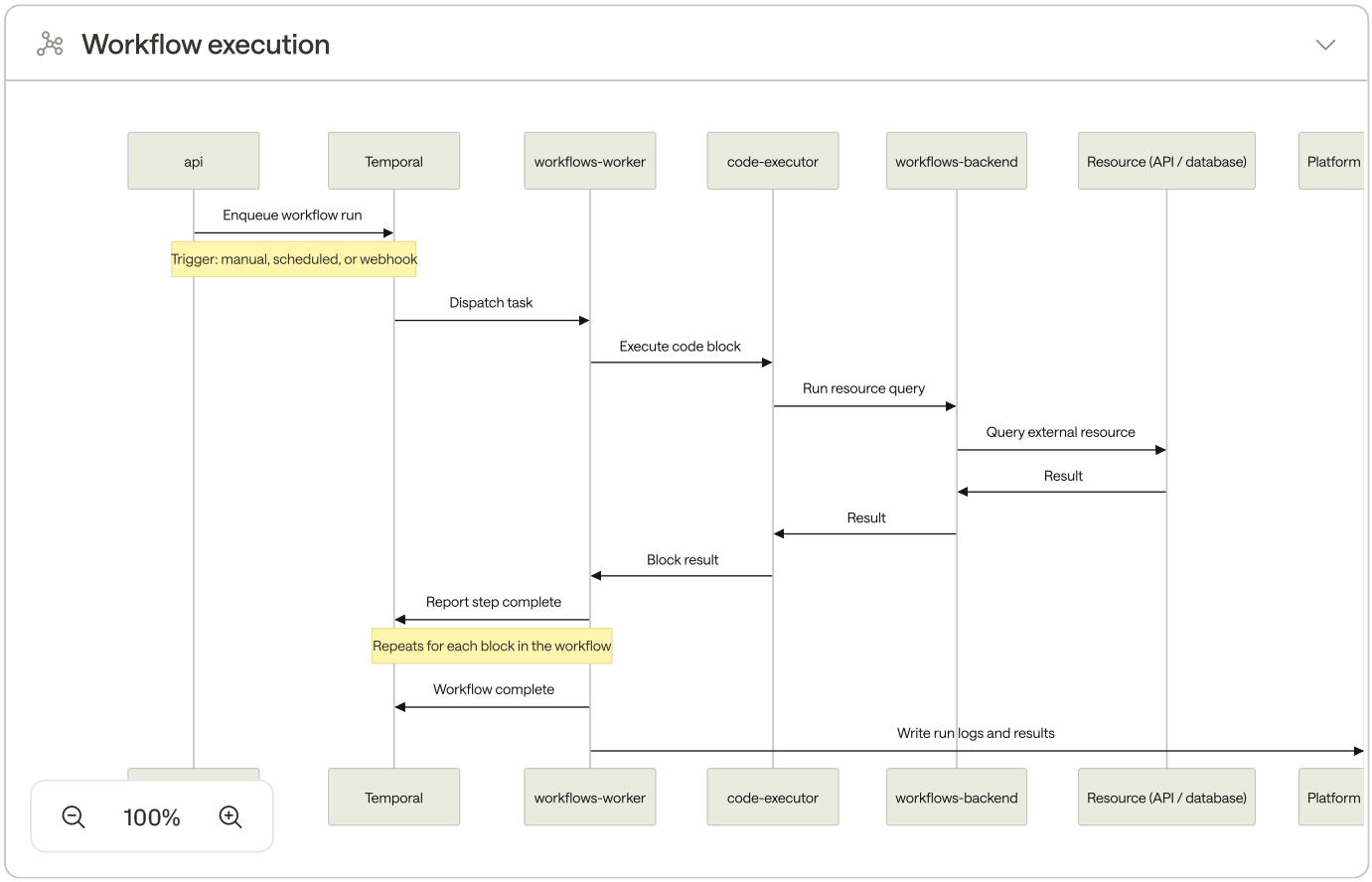
OAuth configuration is required before external MCP clients can authenticate.

Request flows

These diagrams show how requests pass through the deployment for the two main execution paths.







Other dependencies

Self-hosted deployment instances have additional dependencies.

Blob storage

Object storage is required for app storage, Git repository storage, and agent sandbox snapshots. This is the only stateful dependency that is not automatically provisioned — you must configure it via environment variables.

Supported providers:

- Amazon S3 (IAM keys or IAM roles)
- Azure Blob Storage
- Google Cloud Storage (service account keys)

For Docker Compose deployments, a bundled MinIO container provides local blob storage suitable for development and POC environments.

Resources

Each data source you use with Retool is represented by a resource, such as APIs and external databases. Your deployment instance must have access to any of its resources to interact with data.

Temporal

Temporal is a distributed system used to schedule and run asynchronous tasks for Retool Workflows and Retool Agents. A self-hosted Retool instance uses a Temporal *cluster* to facilitate the execution of each workflow amongst a pool of self-hosted workers that make queries and execute code in your VPC. Temporal manages the queueing, scheduling, and orchestration of workflows to guarantee that each workflow block executes in the correct order of the control flow. It does not store any block results by default.

You can use a Retool-managed cluster on Temporal Cloud, which is recommended for most use cases. You can also use an existing self-managed cluster that is hosted on Temporal Cloud or in your own infrastructure. Alternatively, you can spin up a new self-hosted cluster alongside your self-hosted Retool instance.

Docker images

Retool uses Docker images that configure the containers and their respective services. Each container runs one or more services to distribute functionality. This approach allows for efficient scaling of a Retool deployment instance as needs grow.

Retool maintains image repositories on Docker Hub:

- The backend image is used for most containers and services.
- The code-executor-service image is used to execute blocks of code in Retool Workflows.
- The js-executor-service image is used for backend function execution.
- The agent-sandbox-service image is used for the agent sandbox.

retool.example

Each Docker Hub repository tag corresponds to a particular version of Retool. You specify the version to use when deploying or upgrading a release. For instance:

- `tryretool/backend: X.Y.Z-stable`
- `tryretool/code-executor-service: X.Y.Z-stable`

For information about hardened variants of these images, including `arm64` support and security changes, see Hardened images (beta).